计算机组成原理

第2章　习题课

本章小结

- 数值数据，非数值数据，基本运算方法，数据校验
- 数值数据表示要素

✓进位制数

✓机器数-带符号数的表示

✓定点、浮点表示法（表示范围，典型值）

- 机器数、真值
- 原码、反码、补码、移码
- 定点数、浮点数
- 字符表示法、汉字表示法
- 常用数据校验技术：码距$2^r \geq k + r + 1$

1. 常用的几种进位制

（1）2进制：0、1

（2）8进制：0、1、2、…、7

（3）16进制：0、…、9、A、B、C、D、E、F

2. 进制之间的转换

（1）整数 10 → 2 (除2取余法)

（2）小数 10 → 2 (乘2取整法)

（3）整数 2 → 10(按权相加)

（4）小数 2 → 10(按权相加)

（5）16进制↔2进制(逐位转换/分组转换)

(1) $29_{10} \rightarrow X_2$ 除2取余

2 | 29		
2 | 14	→ 1	低位
2 | 7	→ 0	
2 | 3	→ 1	
2 | 1	→ 1	
0	→ 1	高位

$X_2=11101_2$

(2) $0.6875_{10} \rightarrow X_2$ 乘2取整

$0.6875 \times 2 = 1.375$ → 1 高位

$0.375 \times 2 = 0.75$ → 0

$0.75 \times 2 = 1.5$ → 1

$0.5 \times 2 = 1.0$ → 1 低位

$X_2=0.1011_2$

(3) $1101.11_2 \rightarrow X_{10}$ 按权相加

$X_{10}=1\times2^3+1\times2^2+0\times2^1+1\times2^0+1\times2^{-1}+1\times2^{-2}$

$=13.75_{10}$

(4) $111101_2 \rightarrow X_{16}$ 4位分组、按组转换

$X_{16}=111101=0011\ 1101=\underline{0011}\ \underline{1101}=3D_{16}$

(5) $28AF_{16} \rightarrow X_2$ 逐位转换

$X_2=28AF$

$=\underline{0010}\ \underline{1000}\ \underline{1010}\ \underline{1111}$

(6) $28AF_{16} \rightarrow X_{10}$ 按权相加

$X_{10}=28AF=2\times16^3+8\times16^2+10\times16^1+15\times16^0$

$=10415_{10}$

机器码小结---求法及特点

- 4种编码方式
  - 原码　最高位符号位，0为正，1为负，数值位不变（符号人为规定）
  - 反码　正同原，负：符号位同原，其余取反
  - 补码　正同原，负：符号位同原，其余取反，末位+1　或反码末位加1（符号经过模运算求得）
  - 移码　补码符号位取反（只表示整数）

原码表示区间

- 纯小数$x_0.x_1x_2\cdots x_n$
  - $-0.11\cdots1 \sim +0.11\cdots1$
  - $[-(1-2^{-n}), 1-2^{-n}]$ 或 $(-1,1)$
- 纯整数$x_0x_1x_2\cdots x_n$
  - $-011\cdots1 \sim +011\cdots1$
  - $[-(2^n-1), 2^n-1]$ 或 $(-2^n, 2^n)$
- 对称区间

原码特点

表 2.1　原码的表示方法

	真　　值	原　　码
正数	$x=+0.x_1x_2\cdots x_n$	$[x]_{原}=0.x_1x_2\cdots x_n$
负数	$x=-0.x_1x_2\cdots x_n$	$[x]_{原}=1.x_1x_2\cdots x_n$
正数	$x=+x_1x_2\cdots x_n$	$[x]_{原}=0x_1x_2\cdots x_n$
负数	$x=-x_1x_2\cdots x_n$	$[x]_{原}=1x_1x_2\cdots x_n$

✓ 直观易懂：第一位为符号位，其他为数值位

✓ 0的表示不唯一

✓ 加、减运算方式不统一：
加法：符号同数值部分直接加，符号异数值部分减。
减法：先比较两个运算数绝对值大小，符号位同绝对值大的那个数的符号位。

反码特性

- 两个零
- 求反用逻辑门容易实现
- 运算仍然很复杂

补码表示区间

- 定点小数 $.x_1x_2\cdots x_n$
  - $-1.00\cdots0 \sim +0.11\cdots1$
  - $[-1,1-2^{-n}]$ 或 $[-1,1)$
- 定点整数 $x_1x_2\cdots x_n$
  - $-100\cdots0 \sim +011\cdots1$
  - $[-2^n, 2^n-1]$ 或 $[-2^n, 2^n)$
- 非对称区间，左侧多一个数

注：补码相对原码反码少一个负零，多表示一个负一。

负数比正数多一个数

4、原码和补码之间的转换

(1) 已知$[X]_{原}$，求$[X]_{补}$

[例] 已知$[X]_{原}$= 10011010，求$[X]_{补}$

解：$[X]_{原}$= 10011010　原码为负数

↓↓↓↓↓↓↓↓　符号位不变，其余各位变反

11100101

\+ 1　末位加1

$[X]_{补}$=11100110

(2) 已知$[X]_{补}$，求$[X]_{原}$

$$[[X]_{补}]_{补}=[X]_{原}$$

[例] 已知$[X]_{补}=11101100$，求$[X]_{原}$

解：$[X]_{补}=11101100$　　补码为负数

↓↓↓↓↓↓↓↓　　符号位不变，其余各位变反

10010011

\+　　1　　末位加1

$[X]_{原}=10010100$

(3) 求补(变补)，即已知$[X]_{补}$，求$[-X]_{补}$

$[X]_{补}$的代码连同符号位一起变反，末位再加1，即得到$[-X]_{补}$

[例] 已知$[X]_{补}=01010110$，求$[-X]_{补}$

解：$[X]_{补}=0\,1010110$　　不区分正负数

↓↓↓↓↓↓↓↓　　连同符号位一起变反

10101001

\+ 1　　末位加1

$[-X]_{补}=1\,0101010$

(4) 求补(变补)，双符号位补码

$$[X]_{补}=\begin{cases} X & 0\le X<2^{n} \\ 2^{n+2}+X & -2^{n}\le X<0 \end{cases} \quad \text{mod } 2^{n+2}$$

例：00001010110

1101010001

$$[X]_{补}=\begin{cases} X & 0\le X<1 \\ 4+X & -1\le X<0 \end{cases} \quad \text{mod } 4$$

例：00.01010110

11.01010001

符号位01表示正溢出，10表示负溢出，最高位表示正确符号位

机器码小结---机器码表示范围

- n+1位数，数据位n位

	定点整数		定点小数	
原码反码	$[1-2^n, 2^n-1]$	$(-2^n, 2^n)$	$[2^{-n}-1, 1-2^{-n}]$	$(-1, 1)$
补码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	$[-1, 1-2^{-n}]$	$[-1, 1)$
移码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	小数无移码	

二进制代码	无符号数值	原码值	反码值	补码值
0000 0000	0	+0	+0	+0
0000 0001	1	+1	+1	+1
⋮	⋮	⋮	⋮	⋮
0111 1110	126	+126	+126	+126
0111 1111	127	+127	+127	+127
1000 0000	128	-0	-127	-128
1000 0001	129	-1	-126	-127
1000 0010	130	-2	-125	-126
⋮	⋮	⋮	⋮	⋮
1111 1101	253	-125	-2	-3
1111 1110	254	-126	-1	-2
1111 1111	255	-127	-0	-1

机器码小结-1

数值数据表示法---定点数的表示

第一个知识点小结



原码

- 直观易懂第一位为符号位，其他为数值位；
- 正零负零两个零，加、减运算方式不统一。
- 符号相异加法不能直接运算。



补码

- 补唯一的零，定点小数补码相对原码少一个负零，多一个负一；定点整数补码相对原码少一个负零，多一个负数。
- 符号位可以直接参与运算，减法可以变成加法



移码

- 移码与补码的符号位相异，数据位相同，移码用于浮点数阶码。
- 移码保持数据原有大小顺序，便于进行比较操作。

机器码小结-2

- 4种编码方式-求法
  - 原码　最高位符号位，0为正，1为负，数值位不变
  - 反码　正同原，负：符号位同原，其余取反
  - 补码　正同原，负：符号位同原，其余取反，末位+1
  - 移码　补码符号位取反

机器码小结-3

- n+1位数，数据位n位

	定点整数		定点小数	
原码反码	$[1-2^n, 2^n-1]$	$(-2^n, 2^n)$	$[-2^{-n}-1, 1-2^{-n}]$	$(-1, 1)$
补码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	$[-1, 1-2^{-n}]$	$[-1, 1)$
移码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	小数无移码	

机器码小结-4

- 4种编码方式-用法
  - 原码　用来表示浮点（实）数的尾数
  - 反码　已不用于表示数值数据
  - 补码　50年代开始成为整数标准
  - 移码　用于浮点数阶码

练习

1 计算机字长16位，采用补码表示整数，下列关于其表示数据范围的描述中错误的是（　）

C

A.能表示的最小正数是1

B.能表示的最大负数是-1

C.不能表示最小负数- (2 ^15)

D.能表示最大正数是(2^15) – 1

2、计算机内部的定点数大多用补码表示，下列关于补码特点的叙述中正确的是（　　）

B

A.负数不存在补码，正数的补码可用反码加1求得

B.符号位可以和数值部分一起参加运算

C.减法不能用加法来实现

D.零的表示不唯一

练习

- 4　设x=0.1011,求 x的原码、反码和补码。

- 5　设x=-0.1011,求 x的原码、反码和补码。

- 6 将下列数转换成真值

$[x]_{补}$= 1010 0011　x=

$[x]_{补}$ = 10100010 x =

练习

- 4　设x=0.1011,求 x的原码、反码和补码。
- $[x]_{原}$ = 0.1011, $[x]_{反}$ = 0.1011，$[x]_{补}$=0.1011
- 5　设x=-0.1011,求 x的原码、反码和补码。
- $[x]_{原}$ = 1.1011, $[x]_{反}$ = 1.0100，$[x]_{补}$=1.0101
- 6 将下列数转换成真值

$[x]_{补}$= 1010 0011 x =-1011101

[x]原 = 11011101

$[x]_{补}$ = 10100010　x=-1011110

[x]原 = 11011110

回顾2 数的定点表示方法

- 定点表示 (小数点位置固定的数)
  - 定点整数
  - 定点小数
  - 仅能表示纯小数及纯整数

无符号定点整数：略去符号位的正整数称为无符号整数

$0 \sim (2^{n+1}-1)$

有符号和无符号共存时无符号优先

带符号定点整数：可以用原码、反码、补码、移码表示

带符号定点小数：可以用原码、补码、反码表示

带符号的定点数表示范围

定点小数表示范围-以原码为例

典型值	真值	代码序列
最小负数	$-(1-2^{-n})$	1.1…11
最大负数	-2^{-n}	1.0…01
最小非零正数	2^{-n}	0.0…01
最大正数	$1-2^{-n}$	0.1…11

小数：$[-(1-2^{-n}), 1-2^{-n}]$　　或　　$(-1,1)$

带符号的定点数表示范围

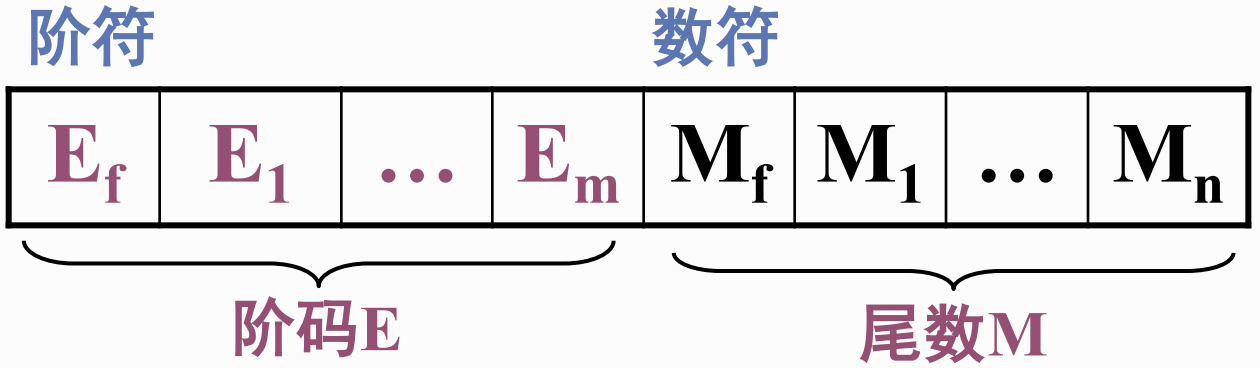
定点整数表示范围-以补码为例

典型值	真值	代码序列
最小负数	-2^n	10…00
最大负数	−1	11…11
最小非零正数	1	00…01
最大正数	2^n-1	01….11

定点整数：$[-2^n, 2^n-1]$　或　　$[-2^n, 2^n)$

③浮点数的机器(存储)格式

浮点数真值： $N = \pm R^{E} \times M$



R： 阶码的底数，隐含约定为2。

E： 阶码，定点整数，补码或移码表示，其位数决定了数值的范围；

M： 尾数，为定点小数，原码或补码表示，其位数决定着数的精度；数符表示数的正负。

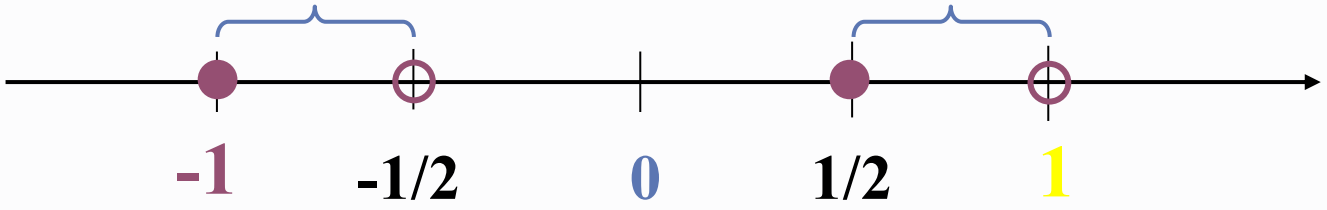
① 浮点数用原码表示时

$1/2 \le |M| < 1$

② 浮点数用补码表示时

$-1 \le M < -1/2$ 或 $1/2 \le M < 1$

机器实际判断补码是否为规格化数时是根据符号位与有效位最高位不同时才是规格化数，-1/2的补码1.10000，-1尾数1.0000



对于原码： 规格化以后尾数的最高有效位为“1”

$M_{原}$=0.1000，1.1010 ~~$M_{原}$=0.0010，1.0110~~ 非规格化

对于补码：

正数，规格化后最高数值位为“1” 如0.1010，0.1110

0.625 0.875

负数，规格化后最高数值位为“0” 如1.0010，1.0000

-0.875 -1

- **尾数的表示**

原$|M|$=0.1××...×

$(0.1000001)_{补}$ = 0.1000001

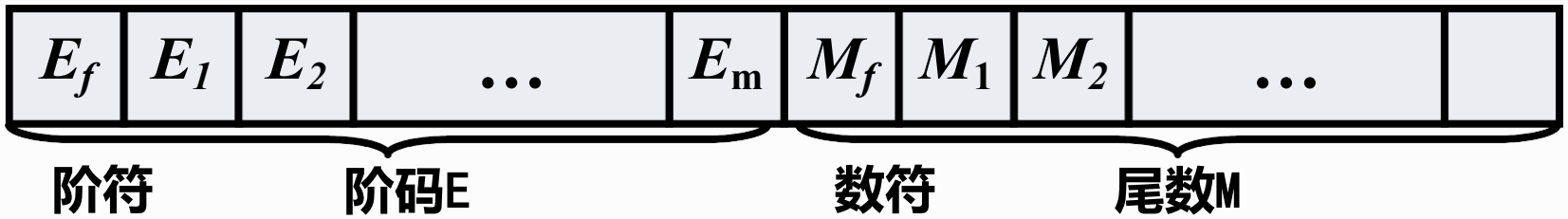
$(-0.1000001)_{补}$ = $1.0111111)_{补}$

对于原码表示的尾数，规格化的特征是：$M_1=1$；

对于补码表示的尾数，规格化的特征是：尾数最高数位和符号位不同。

(1) 浮点表示法

- **真值与浮点数之间的转换**



例　某浮点数格式如图示，字长32其中阶码8位，含一位阶符，补码表示，以2为底；尾数24位，含一位数符，补码表示，规格化。若浮点数代码为（A3680000）$_{16}$求其真值。$N=2^E \times M$

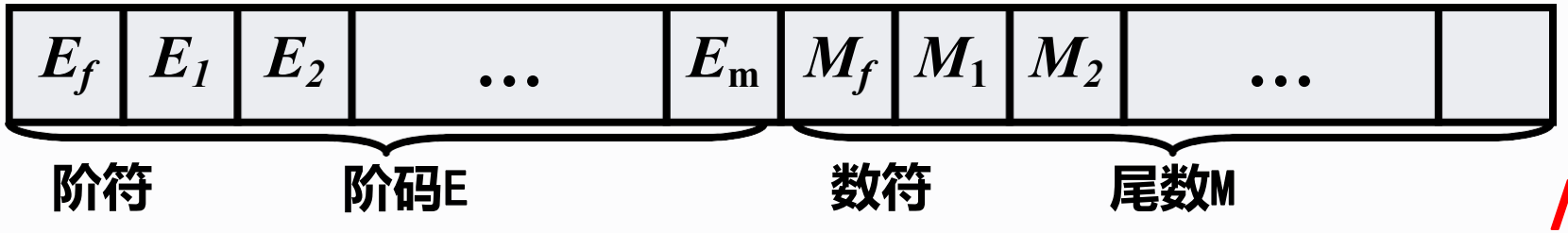
（A3680000）$_{16}$ = （1010 0011, 0110 1000 000…0）$_2$

E= - （1011101）$_2$ = - （93）$_{10}$

M = （0. 11010…0）$_2$ = （0. 8125）$_{10}$

$N = 2^{-93} \times 0.8125$

(1) 浮点表示法



$N=2^E \times M$

例　按上述浮点格式将 - (1011.11010…0)$_2$写成浮点数代码（补码）

N = - (1011.11010…0)$_2$ = - (0.101111010…0)$_2$×2^4

E = (4)$_{10}$ = (00000100)$_2$

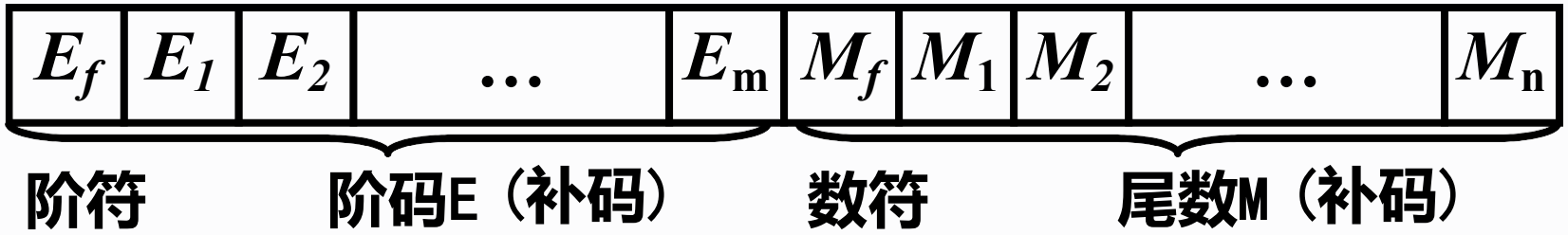
$E_{补}$ = 00000100

$M_{原}$ = - (0.101111010…0)$_2$

$M_{补}$ = (1.010000110…0)$_2$

浮点数代码为 (00000100，1010000110…0)$_2$ = (04A18000)$_{16}$

浮点表示法范围与精度



浮点数标准 IEEE754



- **规格化数**(Normal)

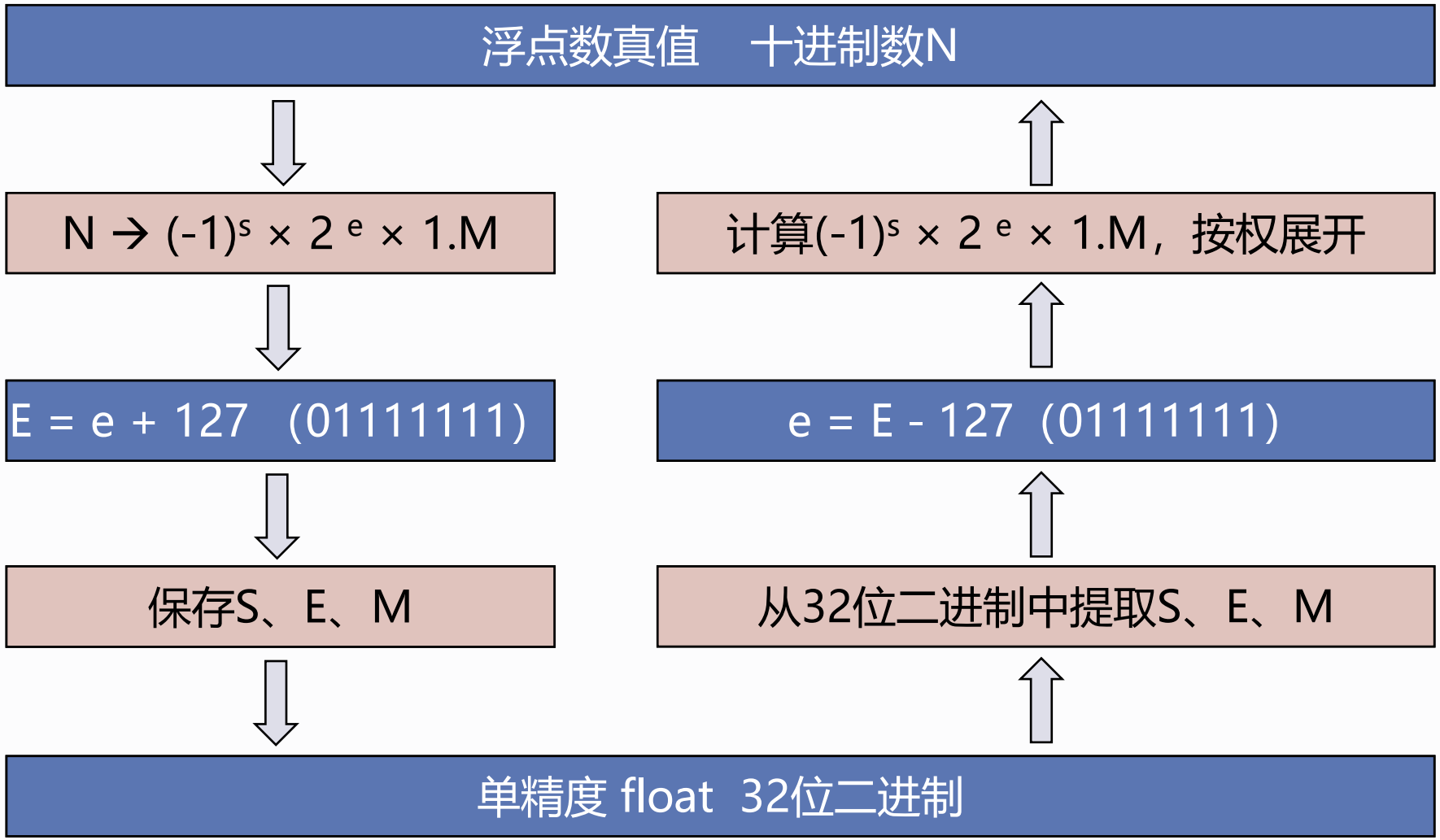
$$(-1)^{s} \times 1.M \times 2^{E-127}$$

- 尾数**原码表示**，表示区间对称
- 规格化数最高数字位恒为1，该位为**隐藏位**，可节约尾数数据位
- E=0，M=0：表示机器零

IEEE754 规格化浮点数表示范围

格式	绝对值最小	绝对值最大值
单精度规格化 $(-1)^{s}\times 1.M\times 2^{E-127}$	$E_{min}=1$, $M=0$, $1.0\times 2^{1-127}=2^{-126}$	$E_{max}=254$, $M=1.1111\ldots1$, $1.1111\ldots1\times 2^{254-127}$ $=2^{127}\times(2-2^{-23})$

float型与真值之间的变换流程



数值数据的浮点数表示--IEEE754

例：将短浮点数36.625转换成IEEE754格式

1　　将36.625转换成二进制形式　36.625=100100.101

2　　非规格化形式转换成规格化形式1.M

100100.101=1.00100101×2^{5}=1.00100101×2^{101}

3　　提取S,E,M

符号位 S=0

阶码　　E=5+127=101+01111111=10000100

尾数　　M=00100101000000000000000

10

数值数据的浮点数表示--IEEE754

- 例：将短浮点数C1C90000转换成十进制数

1　　十六进制→二进制形式，并分离出符号位、阶码和尾数。

- **C1C90000H=**

1;10000011;10010010000000000000000

2　　计算出阶码E真值（移码 - 偏置值2^7）

e=10000011-01111111=100=4 (10)

数值数据的浮点数表示--IEEE754

3　　以规格化二进制数形式写出此数

1.1001001×2^4

4　　写成非规格化二进制数形式　11001.001

5　　转换成十进制数，并加上符号位。

$(11001.001)_2=(25.125)_{10}$

所以，该浮点数=-25.125

（2） 奇偶校验

存储校验技术

2 常用的数据校验码 奇偶校验码

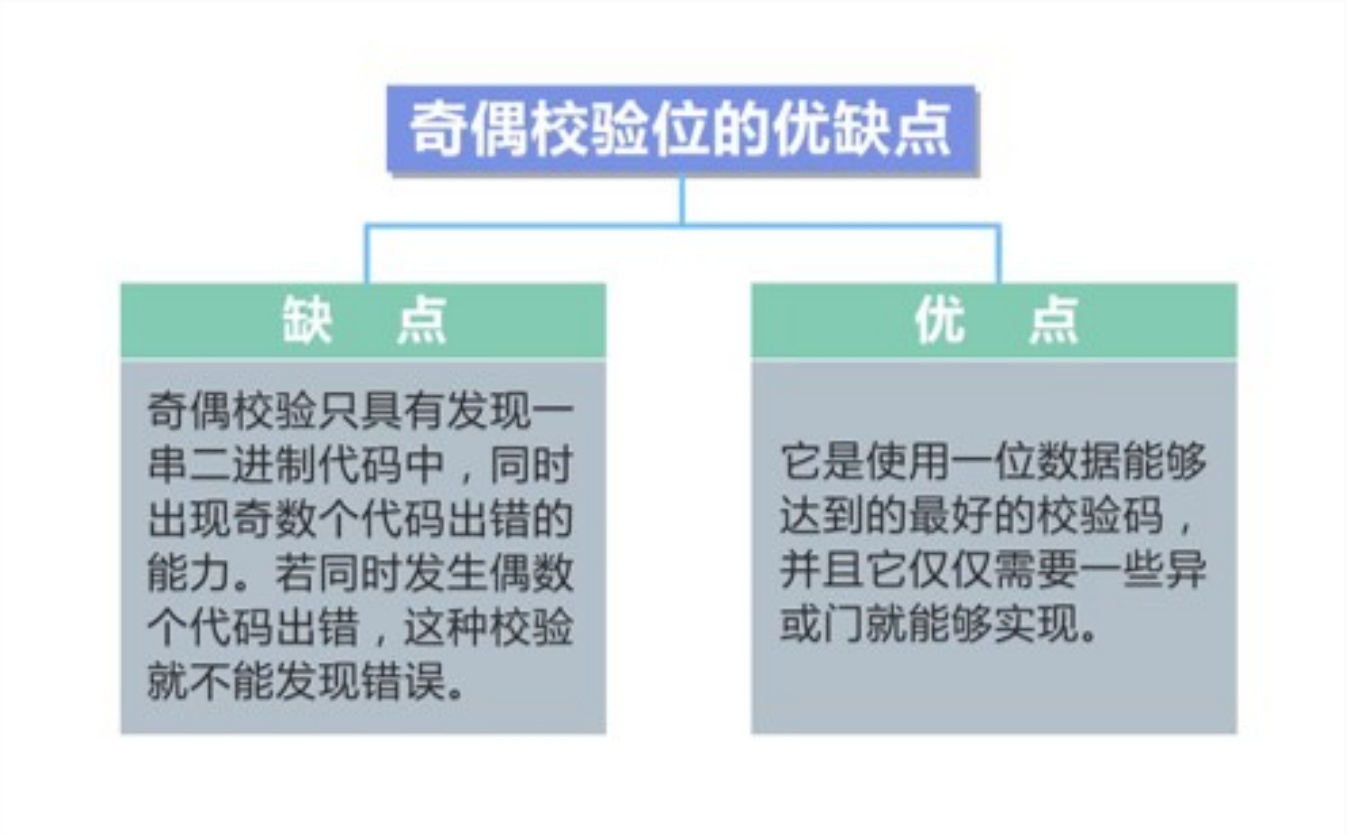
这个校验位取“0”还是取“1”的原则是：

- 若是奇校验，则连同校验位在内编码里含“1”的个数共有奇数个；
- 若是偶校验，则连同校验位在内编码里含“1”的个数是偶数个。

只具有有限的检错功能，而没有纠错的功能。

注意两个概念的不同：奇偶校验位和奇偶校验码的，位只有1位，码是K+1位（数值为位K位）

（2） 奇偶校验


奇偶校验位的优缺点
缺　点
奇偶校验只具有发现一串二进制代码中，同时出现奇数个代码出错的能力。若同时发生偶数个代码出错，这种校验就不能发现错误。
优　点
它是使用一位数据能够达到的最好的校验码，并且它仅仅需要一些异或门就能够实现。

（3）海明校验码

存储校验技术

纠 错 码

- 常用的纠错码有海明码、循环冗余码等。
- 只要使用足够的校验位，就可以达到对任何错误的检测和纠错水平。
- 若需要校正字长 k 位的单错，则必须添加 r 个校验位，它能够指出 k+r 个错误及无错误的情况，r 必须满足 $2^r \geq k+r+1$ 。

r个校验位形成一个指错字，可以指出2^r种状态，其中一种是合法编码，因此，余下的状态就能指出2^r-1位找那个某位出错。

单元测验

1 若某数x的真值为-0.1010，在计算机中该数表示为1.0110，则用的编码方式为（）

A 移码 B 补码 C 反码 D 原码

2 在小数定点机中，下述第（ ）种说法是正确的

A 补码 、反码、原码三种机器数均可表示-1；

B 补码 、反码、原码三种机器数均不可表示-1；

C 反码、原码不可表示-1，补码可表示-1；

D 补码 、反码、原码三种机器数均可表示-1，且三种机器数表示范围相同。

3 在下列机器数中，零的表示形式是唯一的（）

A 补码 B原码 C 反码 D原码和补码

单元测验

4 若浮点数用补码表示，则判断运算结果为规格化数的方法是（）

A 阶符与数符相同

B 阶符与数符不同

C 数符与尾数小数点后第一位数字相同

D 数符与尾数小数点后第一位数字不同

5 假定下列字符码中有奇偶校验位，但没有数据错误，采用偶校验的字符码是（）

A11001011 B 11010111 C 11001101 D 11001011

单元测验

6 计算机字长16位，采用补码表示整数，下列关于其表示数据范围的描述中错误的是（　）

A.能表示的最小正数是1

B.能表示的最大负数是-1

C.不能表示最小负数- (2 ^15)

D.能表示最大正数是(2^15) – 1

7 对于IEEE754格式的浮点数，下列描述正确的是（　）

A.阶码和尾数都用补码表示

B.阶码用移码表示，尾数用补码表示

C.阶码用移码表示，尾数用原码表示

D.阶码和尾数都用原码表示

单元测验

8 定点8位字长的字，采用2的补码形式表示时，一个字所表示的整数范围是（　）

A.-129~+128

B.-128~+127

C.-127~+127

D.-128~+128

9 8位补码10010011等值扩展至16位，其机器数为（　）

A.0000000010010011

B.1111111110010011

C.1000000010010011

D.111111110010011

单元测验

10、计算机内部的定点数大多用补码表示，下列关于补码特点的叙述中正确的是（　　）

A.负数不存在补码，正数的补码可用反码加1求得

B.符号位可以和数值部分一起参加运算

C.减法不能用加法来实现

D.零的表示不唯一

11 用海明码对长度为8位的数据进行检错纠错时，若能纠正一个错，则检验位数至少为（）

A 2 B 5 C 4 D 3

12　设x = -0.1011，则$x_{补}$=

13　设$x_{补}$=1.1011，则 x=

14 将十进制数20.59375转换成32位IEEE 754格式浮点数的二进制格式来存储。

答案

1 若某数x的真值为-0.1010，在计算机中该数表示为1.0110，则用的编码方式为（）B

A 移码 B 补码 C 反码 D 原码

2 在小数定点机中，下述第（）种说法是正确的 C

A 补码 、反码、原码三种机器数均可表示-1；

B 补码 、反码、原码三种机器数均不可表示-1；

C 反码、原码三种机器数不可表示-1，补码可表示-1；

D 补码 、反码、原码三种机器数均可表示-1，且三种机器数表示范围相同。

3 在下列机器数中，零的表示形式是唯一的（） A

A 补码 B原码 C 反码 D原码和补码

4 若浮点数用补码表示，则判断运算结果为规格化数的方法是（） D

A 阶符与数符相同

B 阶符与数符不同

C 数符与尾数小数点后第一位数字相同

D 数符与尾数小数点后第一位数字不同

5 假定下列字符码中有奇偶校验位，但没有数据错误，采用偶校验的字符码是（） B

A11001011 B 11010111 C 11001101 D 11001011

6 计算机字长16位，采用补码表示整数，下列关于其表示数据范围的描述中错误的是（　）

C

A.能表示的最小正数是1

B.能表示的最大负数是-1

C.不能表示最小负数- (2 ^15)

D.能表示最大正数是(2^15) – 1

7 对于IEEE754格式的浮点数，下列描述正确的是（　）　C

A.阶码和尾数都用补码表示

B.阶码用移码表示，尾数用补码表示

C.阶码用移码表示，尾数用原码表示

D.阶码和尾数都用原码表示

单元测验

8 定点8位字长的字，采用2的补码形式表示时，一个字所表示的整数范围是（B　）

A.-129~+128

B.-128~+127

C.-127~+127

D.-128~+128

9 8位补码10010011等值扩展至16位，其机器数为（B　）

A.0000000010010011

B.1111111110010011

C.1000000010010011

D.111111110010011

10、计算机内部的定点数大多用补码表示，下列关于补码特点的叙述中正确的是（　）
B ）

B

A.负数不存在补码，正数的补码可用反码加1求得

B.符号位可以和数值部分一起参加运算

C.减法不能用加法来实现

D.零的表示不唯一

11 用海明码对长度为8位的数据进行检错纠错时，若能纠正一个错，则检验位数至少为（C）

A 2 B 5 C 4 D 3

$$2^r>=r+k+1$$

12 设x = -0.1011，则$x_{补}$= 1.0101

13 设$x_{补}$=1.1011，则 x= -0.0101

14、将十进制数20.59375转换成32位IEEE 754格式浮点数的二进制格式来存储。

解：

首先分别将整数和分数部分转换成二进制数：

20.59375=10100.11011

移动小数点，使其变成1.M的形式：

1.010011*2^4

则

S=0，e=4,E=100+01111111=10000011，M=010010011

最后得到32位浮点数的二进制存储格式为：

0 10000011 010010011 0000 0000 0000 00

2 算术运算

✓ 补码加法：$[X+Y]_{补} = [X]_{补} + [Y]_{补}$

符号位参与运算

和的补码 = 补码的和

注：

1 用补码表示的两个数进行加法运算，结果为补码。

2 符号位与数值位一样参加运算，符号位相加所产生的进位要丢掉。

✓补码减法：$[X-Y]_{补} = [X+(-Y)]_{补} = [X]_{补} + [-Y]_{补} = [X]_{补} - [Y]_{补}$

补码的差=差的补码

减法变加法，关键是求$[-Y]_{补}$

$$[-Y]_{补} = \overline{Y} + 1$$

注

1 用补码表示的两数进行减法运算，其结果仍为补码。

2 符号位与数值位一样参加运算。

3 两数相减，等于被减数加上减数的机器负数。

✓ 溢出检测

单符号位溢出检测1

- 溢出逻辑： **正正得负 负负得正**
- 设两数符号位为 $f_0\ f_1$，和数符号位 f_s
- 溢出检测信号 $Overflow\quad (OF)$

$$Overflow = \bar{f}_0\bar{f}_1 f_s + f_0 f_1 \bar{f}_s$$

当Overflow =1 时，溢出

溢出检测

1. 溢出产生的条件

正数－正数	⇒	**不可能溢出**
负数－负数	⇒	**不可能溢出**
正数－负数	⇒	**可能溢出**
负数－正数	⇒	**可能溢出**

结论： 两异号数相加或两同号数相减一定不会产生溢出，
两同号数相加或两异号数相减时才有可能产生溢出。

［假设］补码表示的A、B两数做加减运算

A：4位尾数，1位符号(S_A)
B：4位尾数，1位符号(S_B)

> 补码，故符号位也参加运算

结果的符号，记为S_f
符号位的进位，记为C_f
尾数最高位的进位，记为C

①硬件判断逻辑一（S_A、S_B与S_f的关系）

溢出逻辑= $\overline{S}_A\overline{S}_BS_f + S_AS_B\overline{S}_f$　1溢出，0无溢出

②硬件判断逻辑二（C_f与C的关系）

溢出逻辑= $C_f \oplus C$　1溢出，0无溢出

③硬件判断逻辑三（双符号位S_{f1}、S_{f2}）

溢出逻辑= $S_{f1} \oplus S_{f2}$

00/11-正确；01-正溢；10-负溢；

补码加法的几种情况

正常结果

$$\begin{array}{r} 0.1\,0\,1\,0\,1 \\ +\ 0.0\,1\,0\,0\,0 \\ \hline 0.1\,1\,1\,0\,1 \end{array}$$

$Overflow1 = \bar{f}_0\bar{f}_1 f_s + f_0 f_1 \bar{f}_s$

$=1\cdot 1\cdot 0 + 0\cdot 0\cdot 1 = 0$

$$\begin{array}{r} 1.1\,0\,1\,0\,1 \\ +\ 1.1\,1\,0\,0\,0 \\ \hline 1\ 1.0\,1\,1\,0\,1 \end{array}$$

模数舍去

正常结果

$Overflow2 = \bar{f}_0\bar{f}_1 f_s + f_0 f_1 \bar{f}_s$

$=0\cdot 0\cdot 1 + 1\cdot 1\cdot 0 = 0$

正正得负

正溢出

$$\begin{array}{r} 0.1\,0\,1\,0\,1 \\ +\ 0.1\,1\,0\,0\,0 \\ \hline 1.0\,1\,1\,0\,1 \end{array}$$

$Overflow3 = \bar{f}_0\bar{f}_1 f_s + f_0 f_1 \bar{f}_s$

$= 1\cdot 1\cdot 1 + 0\cdot 0\cdot 0 = 1$

$$\begin{array}{r} 1.0\,0\,1\,0\,1 \\ +\ 1.1\,1\,0\,0\,0 \\ \hline 1\ 0.1\,1\,1\,0\,1 \end{array}$$

负负得正

负溢出

$Overflow4 = \bar{f}_0\bar{f}_1 f_s + f_0 f_1 \bar{f}_s$

$= 0\cdot 0\cdot 0 + 1\cdot 1\cdot 1 = 1$

单符号溢出检测方法2

C_f=0,C_n=0

```
  0 .1 0 1 0 1
+ 0 .0 1 0 0 0
---------------
  0 .1 1 1 0 1
```

正常结果

C_f=1,C_n=1

```
  1 .1 0 1 0 1
+ 1 .1 1 0 0 0
---------------
1 1 .0 1 1 0 1
```

正常结果

C_f=0,C_n=1

```
  0 .1 0 1 0 1
+ 0 .1 1 0 0 0
---------------
  1 .0 1 1 0 1
```

正溢出

C_f=1,C_n=0

```
  1 .0 0 1 0 1
+ 1 .1 1 0 0 0
---------------
1 0 .1 1 1 0 1
```

负溢出

符号位进位位C_f，最高位进位位C_n

$$Overflow = C_f \oplus C_n$$

符号进位与最高位进位同步时，不溢出。不同步，则溢出

双符号溢出检测方法

```
    0 0 .1 0 1 0 1
+   0 0 .0 1 0 0 0
-------------------
    0 0 .1 1 1 0 1
```

未溢出

```
    1 1 .1 0 1 0 1
+   1 1 .1 1 0 0 0
-------------------
  1 1 1 .0 1 1 0 1
```

未溢出

```
    0 0 .1 0 1 0 1
+   0 0 .1 1 0 0 0
-------------------
  0 1 .0 1 1 0 1
```

正溢出

```
    1 1 .0 0 1 0 1
+   1 1 .1 1 0 0 0
-------------------
  1 1 0 .1 1 1 0 1
```

负溢出

$$Overflow = S_{f1} \oplus S_{f2}$$

符号位01表示正溢出，10表示负溢出，最高位表示正确符号位

✓定点乘法运算

移位、舍入

原码一位乘法：每次用一位乘数乘以被乘数

1. 运算法则

假设被乘数$[X]_{原}=X_sX_1X_2.....X_{n-1}X_n$，乘 数$[Y]_{原}=Y_sY_1Y_2...Y_{n-1}Y_n$乘积$[Z]_{原}=Z_sZ_1Z_2.....Z_{2n-1}Z_{2n}$，则有

$$[Z]_{原}= (X_s \oplus Y_s).(|X| \times |Y|)$$

2. 运算规则

由于机器字长通常仅有n位，两个n位数相乘结果为2n位；且仅仅有两个操作数相加的加法器，难以胜任将n个位积一次相加起来。因此，在计算机中不能直接照搬手工计算乘法的方法，必须进行修改使其便于计算机实现。

1　被乘数和乘数均取绝对值参加运算，符号位单独考虑；

2　被乘数取双符号位，部分积的长度同被乘数，初值为0；

3　从乘数的最低位Y_n开始判断：

若$Y_n = 1$，则部分积加上被乘数$|X|$，然后右移一位；

若$Y_n = 0$，则部分积加上0，然后右移一位。

4　重复第3步操作n次

补码一位乘法

1. 计算公式

假设被乘数$[X]_{补}=X_sX_1X_2……X_{n-1}X_n$，乘 数$[Y]_{补}=Y_sY_1Y_2…Y_{n-1}Y_n$

则有

$$X \times Y = [X]_{补} \times (0.Y_1Y_2 \cdots Y_n) + [-X]_{补} \times Y_s$$

为了对正数和负数都有一致的算法，由英国的Booth 夫妇提出了补码一位乘法的Booth算法。

2. 运算规则

1　符号位参加运算，运算的数均以补码表示；

2　被乘数一般取双符号位参加运算；

3　乘数可取单符号位；

4　乘数末位增设附加位Y_{n+1}，且初值为0。

Y_n (高位)	Y_{n+1} (低位)	操作
0	0	部分积右移一位
0	1	部分积加$[X]_{补}$，右移一位
1	0	部分积加$[-X]_{补}$，右移一位
1	1	部分积右移一位

5　从低位到高位，依次判断相邻的两位乘数的状态，根据比较结果按右表操作。

6 按照上述算法进行n+1步累加操作，n步右移操作。

最后一步不移位，仅根据Y_0与Y_1的比较结果作相应的运算。

3. 原码和补码一位乘的区别

1　前者符号位单独处理，后者符号位参加运算

2　前者只用乘数的末位进行判断，后者乘数增加一位附加位Y_{n+1},用最后两位进行判断

3 前者循环次数，移位次数为n次，后者循环次数n+1,但最后一次不移位

✓定点除法运算

原码一位除法

1. 运算法则

假设被除数$[X]_{原}=X_sX_1X_2.....X_{n-1}X_n$，除数$[Y]_{原}=Y_sY_1Y_2...Y_{n-1}Y_n$

商$[Q]_{原}=Q_sQ_1Q_2.....Q_n$，则有

$$[Q]_{原}= (X_s⊕Y_s).(|X|/|Y|)$$

原码恢复余数法



恢复余数法法则

被除数或余数减除数比较（$+[-Y]_{补}$）：

结果为正，商上1，左移一位

结果为负，商上0，加除数（$+[Y]_{补}$），左移一位

重复上述操作，直到商的位数符合要求为止

3 不恢复余数法

不恢复余数法法则

被除数或余数减除数比较（$+[-Y]_{补}$）：

结果为正，商上1，左移一位，减除数（$+[-Y]_{补}$）

结果为负，商上0，左移一位，加除数（$+[Y]_{补}$）

重复上述操作n+1次

✓ 浮点运算

1 浮点数加减法步骤

- 对阶--小向大看齐
  - 阶码
- 尾数运算
- 规格化
  - 规格化规则
- 舍入
- 溢出判断

注： 若参与运算的浮点数阶码和尾数用补码表示，则采用双符号位，便于溢出判断和尾数规格化处理

浮点数乘法运算

如：$X=2^{m} \times M_x$　　$Y=2^{n} \times M_y$

$$X \times Y = (2^{m} \times M_x) \times (2^{n} \times M_y) = 2^{m+n} \times (M_x \times M_y)$$

1. 阶码相加

阶码相加可能产生溢出，要进行溢出判断，如溢出计算机要进行处理

2. 尾数相乘

尾数相乘可得积的尾数，可按定点乘法运算方法运算

3. 结果规格化

可按浮点加/减法运算规格化方式处理，舍入方式也相同

浮点数除法运算

如：$X=2^{m} \times M_x$　　$Y=2^{n} \times M_y$

$$X \times Y = (2^{m} \times M_x) \div (2^{n} \times M_y) = 2^{m-n} \times (M_x \div M_y)$$

1. 尾数调整

如被除数尾数大于除数尾数 (绝对值)，则将被除数尾数右移一位，阶码+1

2. 阶码求差

商的阶码等于被除数的阶码减去除数的阶码

3. 尾数相除

以被除数的尾数除以除数的尾数以获得商的尾数，尾数相除与定点除法运算相同

单元测验二

1 两个补码相加，采用单符号位，则当（ ）时，表示结果溢出

A 最高位有进位

B 最高位进位和次高位进位异或结果为1

C 最高位进位和次高位进位异或结果为0

D 最高位为1

2 当定点运算发生溢出时，应进行（ ）

A 向左规格化

B 发出出错信息

C 舍入处理

D 向右规格化

3、计算机中的定点二进制运算，减法运算一般通过（ ）实现

A．原码运算的二进制减法器

B．补码运算的二进制减法器

C．原码运算的十进制加法器

D．补码运算的二进制加法器

4.在原码一位乘中，（）

A 设置一位判断位　B 设置两位判断位

C 不设置判断位　D 设置三位判断位

5 计算机字长为n位，下列关于补码一位乘法操作过程中，正确的是（）

A 乘法过程中共执行n次加法和n-1部分积右移

B 乘法过程中共执行n-1次右移和n-1次加法运算

C 乘法过程中，部分积加0，$[x]_{补}$还是$[-x]_{补}$，取决于此时的YnYn+1

D 乘法过程中，右移部分积的目的是为了使部分积与下次的加数对齐。

6 单符号位补码表示的两个同号数相加或异号数相减时，所得结果的符号位与进制位（）运算为1时，结果溢出

A 与

B 非

C 异或

D 或

7 以下各机器数中，引入（）数据表示可消除减法操作

A 补码 B 原码 C 反码 D 移码

8. 设两数相加，符号进位为CF ，最高数值位进位 C，溢出检测信号 Overflow (OF)。若X=0.1011，Y=0.1100，求:

(1)请给出利用符号进位和数值进位实现溢出检测的逻辑表达式。

（2）计算 $[X]_{补}+[Y]_{补}$并根据上述逻辑表达式判断是否溢出。

单元测验二答案

1 两个补码相加，采用单符号位，则当（B）时，表示结果溢出

A 最高位有进位

B 最高位进位和次高位进位异或结果为1

C 最高位进位和次高位进位异或结果为0

D 最高位为1

2 当定点运算发生溢出时，应进行（B）

A 向左规格化

B 发出出错信息

C 舍入处理

D 向右规格化

3、计算机中的定点二进制运算，减法运算一般通过（D ）实现

A．原码运算的二进制减法器

B．补码运算的二进制减法器

C．原码运算的十进制加法器

D．补码运算的二进制加法器

4.在原码一位乘中，（A）

A 设置一位判断位　B 设置两位判断位

C不设置判断位　D 设置三位判断位

5 计算机字长为n位，下列关于补码一位乘法操作过程中，正确的是（ACD）

A 乘法过程中共执行n次加法和n-1部分积右移

B 乘法过程中共执行n-1次右移和n-1次加法运算

C 乘法过程中，部分积加0，$[x]_{补}$还是$[-x]_{补}$，取决于此时的YnYn+1

D 乘法过程中，右移部分积的目的是为了使部分积与下次的加数对齐。

6 单符号位补码表示的两个同号数相加或异号数相减时，所得结果的符号位与进制位（C）运算为1时，结果溢出

A 与 B 非

C异或 D 或

7 以下各机器数中，引入（A ）数据表示可消除减法操作

A 补码 B 原码 C反码 D 移码

8. 设两数相加，符号进位为CF ，最高数值位进位 C，溢出检测信号 Overflow (OF)。若X=0.1011，Y=0.1100，求:

(1)请给出实现溢出检测的逻辑表达式。

（2）计算 $[X]_{补}+[Y]_{补}$并判断是否溢出。

END